



PyTorchSim 의 TensorFlow 확장

학 번: 20220312
이 름: 박준혁
연구 지도교수: 김광선

연구 목적 (Problem statement)

최근 인공지능 모델의 복잡도가 기하급수적으로 증가함에 따라, 이를 효율적으로 처리하기 위한 신경망 처리 장치(NPU, Neural Processing Unit)의 아키텍처 연구 및 하드웨어-소프트웨어 공동 설계의 중요성이 대두되고 있다. 이러한 연구를 뒷받침하기 위해 사이클 수준의 정확도를 제공하는 시뮬레이터의 존재는 필수적이며, 현재 POSTECH PSAL Lab 에서 구축한 NPU 시뮬레이터인 PyTorchSim[1]은 PyTorch[2] 생태계에 강하게 결합되어 있어, 범용적인 활용에 제약이 따르고 있다.

인공지능 연구 및 산업계는 PyTorch 뿐만 아니라 TensorFlow [3], JAX[4] 등 다양한 딥러닝 프레임워크를 혼용하여 사용하고 있다. 새로운 프레임워크에서 작성된 모델을 검증하기 위해 타겟 아키텍처에 맞춘 백엔드 코드 생성기나 런타임을 매번 완전히 새롭게 개발하는 것은 막대한 시간과 개발 비용을 소모하게 만든다. 따라서, 본 연구는 개별 프레임워크에 종속되지 않는 범용적인 중간 표현(IR, Intermediate Representation)인 HLO(High-Level Optimizer)를 매개로 활용하여, TensorFlow 환경을 기존 PyTorchSim 구조에 매끄럽게 통합하고자 한다. 이를 통해 단일 시뮬레이터 인프라로 다중 프레임워크를 지원하는 고도화된 연구 환경을 구축하고자 한다.

연구 배경 (Motivation and background)

현대 딥러닝 프레임워크들은 각자의 고유한 컴파일 파이프라인과 중간 표현을 통해 연산을 최적화하고 하드웨어에 배포하고 있다. PyTorch 의 경우, PyTorch 2.0 부터 도입된 Torch FX Graph 의 형태로 파이썬 수준의 모델 연산 그래프를 포착(Capture)하고, 이를

TorchInductor Backend 에서 연산을 Fusion 하고 C++, Triton 코드로 변환(Lowering)하는 구조를 가진다. PyTorchSim 에서는 Custom TorchInductor Backend 를 구현, NPU 시뮬레이터 특화 MLIR 로 Lowering 하는 부분이 구현되어 있다.

반면 TensorFlow 의 경우 GraphDef 의 형태로 파이썬 모델의 그래프를 캡처하고, XLA(Accelerated Linear Algebra) 컴파일러를 사용해 머신 코드로 변환한다. 이 과정의 결과물은 Machine-Independent 한 HLO(High-Level Optimizer) 코드이며 이후 Backend 를 통해 머신 코드로 변환한다. 이에 본 연구에서는 HLO 코드로부터 기존의 NPU 시뮬레이터 특화 MLIR 로의 커스텀 패스를 구현하고자 한다.

본 연구에서는 TensorFlow 생태계에서 새로운 하드웨어 가속기를 연동하기 위한 방법 중 PJRT(Pretty much Just another RunTime)[5]를 채택한다. PJRT 는 ML 컴파일러가 디바이스와 통신하기 위해 설계된 훨씬 모던하고 통합된 C API 인터페이스이며, XLA 컴파일 파이프라인과 디바이스 백엔드 사이의 명확한 추상화 계층을 제공하고 있다. 이를 사용해 HLO 연산 그래프를 '가로채고', 기존의 MLIR 로의 패스의 가장 적합한 진입점을 확보한다. 또한 PJRT 는 JAX 나 PyTorch/XLA 등 다른 프레임워크에서도 범용적으로 채택되고 있는 표준 인터페이스이므로, 향후 PyTorchSim 의 확장성을 극대화할 수 있다.

연구 방법 (Research proposal)

본 연구는 HLO 와 PJRT 를 활용하여 TensorFlow 파이프라인과 PyTorchSim 을 통합하기 위해 다음과 같은 네 가지 핵심 단계로 나뉘어 연구를 수행하고 있다.

1. HLO 파싱 구현

TensorFlow 모델이 XLA 를 통해 컴파일되는 과정에서 생성되는 HLO 구조를 면밀히 분석하고, 이를 시뮬레이터가 이해할 수 있는 형태로 파싱(Parsing)하는 모듈을 개발하여 기존 PyTorchSim 의 연산 체계와 대응시킨다.

2. PJRT Plugin 구현 및 연결

TensorFlow 컴파일러 파이프라인에 개입하기 위해 PJRT 플러그인을 개발하고 이를 TensorFlow 런타임에 커스텀 백엔드로 등록(Register)하여, 모델 실행 시 컴파일러가 생성한 HLO 그래프가 기본 하드웨어가 아닌 본 연구의 플러그인으로 라우팅되도록 한다.

3. PyTorchSim 과 연결 및 디버깅

PJRT 플러그인을 통해 전달받고 파싱된 HLO 연산들은 Machine-Independent 한 특성을 지니므로, PyTorchSim 의 아키텍처인 Scheduler Codegen 과 ExtensionOverrides 메커니즘 등에 1:1 로 매핑하여 타일링(Tiling) 및 스케줄링 전략을 주입한다.

4. 성능 테스트 및 기존 시뮬레이터와 비교

구현된 통합 시스템의 검증을 위해 CNN, Transformer, LLM 모델 등 대표적인 딥러닝 벤치마크 모델을 구동하고, 실행 정확도/사이클 수/메모리 사용량을 비교 분석하여 전체적인 시뮬레이션의 신뢰성과 효율성을 검증한다.

기대 효과 (Expected output)

본 연구를 통해 딥러닝 프레임워크와 하드웨어 시뮬레이터 간의 종속성을 끊어낼 수 있을 것이다. 특히 현재 훌륭하게 구현된 단일 NPU 시뮬레이터(PyTorchSim)만으로 PyTorch 와 TensorFlow 를 포함해 OpenXLA 생태계가 지원하는 모든 을 모두 수용할 수 있게 되어, 새로운 NPU 아키텍처 설계 및 검증에 소요되는 막대한 소프트웨어 스택 개발 비용을 절감할 수 있을 것이다.

연구 추진 일정

기간	내용	비고
3/20~4/5	HLO 파싱 구현	-
4/6~4/24	PJRT Plugin 구현 및 연결	4/24 중간보고서 제출
4/25~5/8	PyTorchSim 과 연결, 디버깅	5/4 중간발표 녹화 5/8 중간발표 동료평가
5/8~5/29	성능 테스트 및 기존 시뮬레이터와 비교	5/29 최종발표, 최종 보고서

참고 문헌

- [1] Yang, W. a. (2025). PyTorchSim: A Comprehensive, Fast, and Accurate NPU Simulation Framework. Proceedings of the 58th IEEE/ACM International Symposium on Microarchitecture, 1363-1380. doi:10.1145/3725843.3756045
- [2] Jason Ansel, Edward Yang, Horace He, Natalia Gimelshein, Animesh Jain, Michael Voznesensky, Bin Bao, Peter Bell, David Berard, Evgeni Burovski, Geeta Chauhan, Anjali Chourdia, Will Constable, Alban Desmaison, Zachary DeVito, Elias Ellison, Will Feng, Jiong Gong, Michael Gschwind, Brian Hirsh, Sherlock Huang, Kshiteej Kalambarkar, Laurent Kirsch, Michael Lazos, Mario Lezcano, Yanbo Liang, Jason Liang, Yinghai Lu, C. K. Luk, Bert Maher, Yunjie Pan, Christian Puhersch, Matthias Reso, Mark Saroufim, Marcos Yukio Siraichi, Helen Suk, Shunting Zhang, Michael Suo, Phil Tillet, Xu Zhao, Eikan Wang, Keren Zhou, Richard Zou, Xiaodong Wang, Ajit Mathews, William Wen, Gregory Chanan, Peng Wu, and Soumith Chintala. 2024. PyTorch 2: Faster Machine Learning Through Dynamic Python Bytecode Transformation and Graph Compilation. In Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2 (ASPLOS '24), Vol. 2. Association for Computing Machinery, New York, NY, USA, 929–947. <https://doi.org/10.1145/3620665.3640366>
- [3] Martín Abadi, Paul Barham, Jianmin Chen, Zhifeng C. (2016). TensorFlow: a system for Large-Scale machine learning. “12th USENIX symposium on operating systems design and implementation”, 265-283.
- [4] Frostig, R., Johnson, M. J., & Leary, C. (2019, March). Compiling machine learning programs via high-level tracing. In SysML conference 2018.
- [5] <https://openxla.org/xla/pjrt>